

1 Renderer 2

2 Indirect Rendering

In the new renderer, the main passes have been structured around gpu-driven rendering. The current design uses the traditional geometry rasterization pipeline (see Section 2.4), but dispatches draw-calls from the GPU instead. This allows for more flexible rendering and controlling draws from GPU.

The biggest goal in this design was to support culling and LOD selection on the GPU, reducing CPU overhead.

Pipeline changes and state changes are conventionally not supported directly on the GPU. But draw calls can be produced by using `VkDrawIndexedIndirect`. This reads draw calls from a GPU buffer, meaning that only a single `VkDrawIndexedIndirect` command from the CPU needs to be submitted.

To support this, the renderer needs to reduce state changes and pipeline changes. This is done by

- Using a single vertex and index buffer
- Splitting objects into batches (a batch is a group of objects that share the same pipeline)

To support this, a mirror of the scene state needs to be present at all times on the GPU. We call this the *object buffer*. An object keeps reference to its mesh, its material and its transformation. When the scene changes on the CPU, the necessary updates need to happen on the GPU as well.

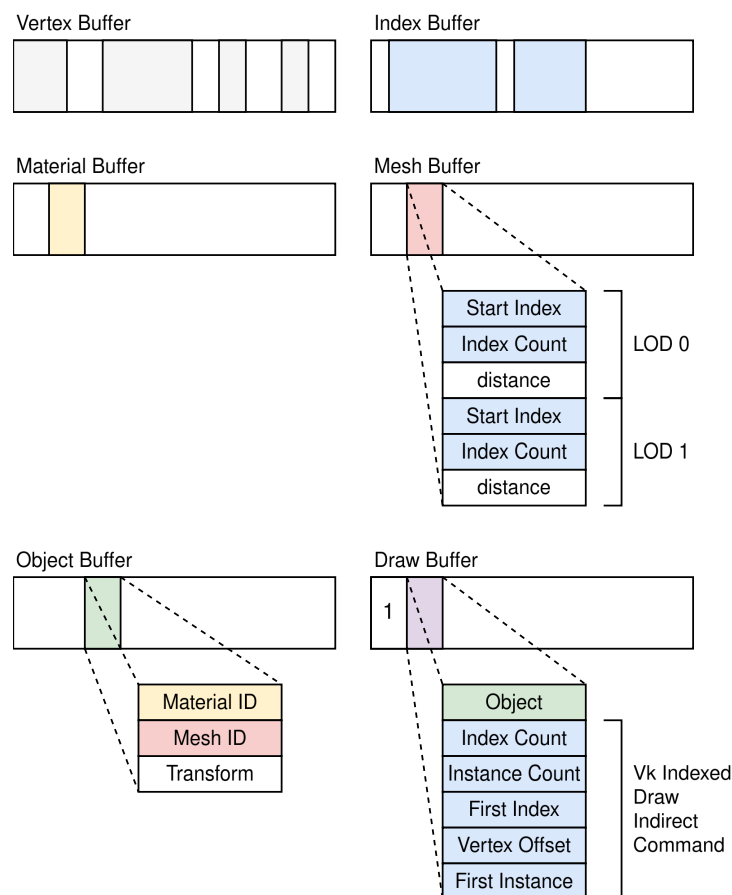


Figure 1: Indirect Rendering

2.1 The Mesh Pass

We call a full draw of the scene a *mesh pass*. There can be multiple mesh passes, like the main forward pass, a directional shadow pass, etc. A mesh pass is first an invocation of the culling

compute shader, generating elements in the *draw buffer*. The draw buffer is later consumed by the *forward indirect* pass, which performs one *VkDrawIndexedIndirect* per batch.

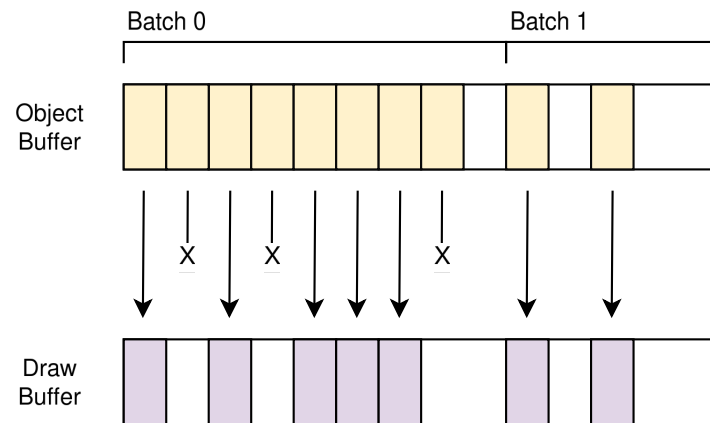


Figure 2: Culling Compute Shader

2.2 Compacting the Draw Buffer

After culling, some objects will be removed and cause holes in the draw buffer. These are not drawn, as their instance count is set to zero. But as the buffer is consumed as draw calls, they will still create overhead. On a GTX 1070, the overhead of drawing 125k objects all being culled is around 2.97ms (average 23ns per object).

Compaction needs to be handled when inserting the objects into the draw buffer. The solution is inspired by [1] and utilizes workgroup ballot operations and parallel prefix sum. The algorithm works on subgroups. Firstly, each invocation checks if the object is visible or not. This value is stored in a ballot for this subgroup. Then a local offset is calculated by counting the number of bits set in the subgroup ballot up to this invocation. The total number of bits set in the ballot is calculated. The first invocation is elected and performs an atomic add to the draw buffers count field, and then broadcast the returned offset (base offset) to the subgroup. Each invocation now sums the base offset with the local offset to retrieve the final offset in the draw buffer. All visible objects can now write to their correct position in the draw buffer. The algorithm is presented in Figure 3.

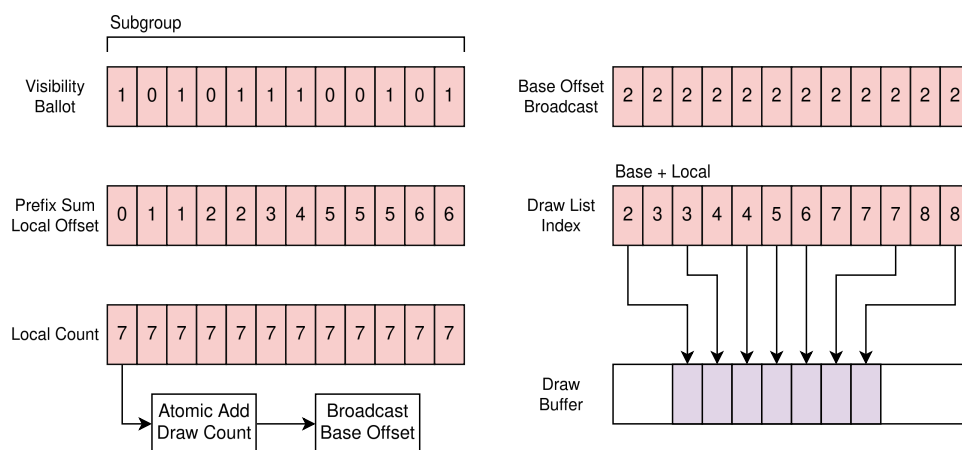


Figure 3: Compaction Algorithm

2.3 Manual Vertex Pulling

A problem with the current design is that a batch requires all object to use the same vertex layout. While not an issue for us, it is a limitation that could be avoided. The whole Input Assembly stage

can be avoided by manually fetching vertex data in the vertex shader. Instead of using a vertex buffer, the vertices can be passed in a storage buffer, and the batches just store the index into it. Each batch specifies its own stride.

This should be trivial to implement, and would allow for more flexible rendering.

2.4 A note on Mesh Rendering

Mesh rendering replaces the old rasterization pipeline, and replaces Input Assembler and Vertex stage with a compute-like shader. This shader can directly output to the rasterizer. A feature of mesh shading is that the mesh shader directly works in workgroups, meaning that it is critical for the application to split meshes into smaller parts (64/128 triangles) called *meshlets*. Culling can be done cheaper than an AABB frustum test if the meshlets are constructed convexly.

Using mesh rendering instead of the current system would have been simpler and likely more efficient. We wouldn't need the current draw buffer. Culling could be implemented on a meshlet level instead of objects, meaning that the geometry would be more evenly distributed.

Overall, mesh shading would be really interesting to try out.

Bibliography

- [1] G. Wihlidal, "Optimizing the graphics pipeline with compute," in *Game Developers Conference*, 2016.